

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

Victor Hugo Cano Siveri

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 07/10/2026

Sumário

1	Introdução	2
2	Metodologia	2
3	Quadro de entregas	3
4	Atividades realizadas	3
4.1	Atividade 1 — Agenda telefônica em Flask	3
4.2	Atividade 2 — Catálogo de filmes — Tom Hanks	4
4.3	Atividade 3 — Desacoplando o login — microsserviço de autenticação	5
4.4	Atividade 4 — Controle de acesso por papel — RBAC	7
4.5	Atividade 5 — Logs e auditoria	9
4.6	Atividade 6 — Upload e perfil de usuário	10
5	Atividades extras (opcional)	12
5.1	Atividade E1 — Documentação Swagger/OpenAPI	12
6	Considerações finais	14
7	Declaração de autoria	15

1 Introdução

A disciplina de Introdução à Computação em Nuvem, integrante da formação em Sistemas Inteligentes, tem como propósito ensinar os conceitos fundamentais de infraestrutura e arquitetura de nuvem a partir de vivências práticas em ambientes reais de produção. Em vez de limitar-se à teoria, o curso colocou os estudantes diretamente no papel de usuários de uma infraestrutura gerenciada: cada aluno dispôs de um banco de dados relacional próprio com credenciais exclusivas, além de um subdomínio individual e portas dedicadas para publicação de serviços. Essa configuração serviu de base para a discussão e aplicação contínua de temas centrais da computação em nuvem, tais como multi-tenancy (múltiplos clientes compartilhando o mesmo servidor físico), isolamento de acessos e recursos, gestão segura de credenciais e os modelos de responsabilidade compartilhada entre usuário e infraestrutura.

Como fio condutor prático ao longo do bimestre, foi desenvolvido e aprimorado o projeto de uma plataforma web de catálogo de filmes do ator Tom Hanks, integrada à API pública do TMDb (The Movie Database). O projeto foi planejado para vivenciar a transição de um sistema web tradicional para uma arquitetura em nuvem escalável e segura: partindo de um monólito em container único, a solução evoluiu para uma topologia distribuída baseada em microsserviços containerizados via Docker. Essa jornada envolveu o desacoplamento de responsabilidades centrais, como a criação de um serviço isolado de autenticação e recuperação de credenciais (auth-service), a implementação de controle de acesso baseado em papéis (Role-Based Access Control - RBAC), um serviço independente de auditoria em memória com Redis Streams (log-service) e o armazenamento externo de imagens de perfil via Object Storage (MinIO).

Este relatório documenta detalhadamente todo o processo de engenharia, escolhas de design e resultados obtidos na construção dessa infraestrutura. O documento está organizado em seções cronológicas e temáticas que detalham: a arquitetura de rede interna e os mecanismos de segurança adotados entre os containers; os fluxos de autenticação, recuperação de senha e autorização com RBAC; a estratégia de logs transacionais de alta performance; e a integração com armazenamento desacoplado de objetos para mídias de perfil. Por fim, são apresentadas as rotinas de teste, instruções de execução do ambiente e evidências do correto funcionamento de cada componente.

2 Metodologia

Todas as atividades foram desenvolvidas de forma iterativa no mesmo repositório público do GitHub (fezinhopatta/Tom-Hanks-api), utilizando ramificações (**branches**) específicas para estruturar a evolução arquitetural do sistema a partir do monólito inicial em Flask até à sua divisão em microsserviços.

O ambiente de execução e orquestração baseou-se em contentores Docker geridos via `docker-compose`, interligados através de uma rede interna do tipo **bridge** (app-network). Foram integradas tecnologias complementares ao longo do projeto: o consumo da API pública do TMDb para dados cinematográficos, persistência relacional no MariaDB externo, envio transacional de mensagens eletrônicas com Brevo (e Mailtrap em ambiente de testes), armazenamento em memória com Redis Streams para auditoria e registo de eventos no log-service, gestão de ficheiros estáticos/avatars com recurso ao Object Storage MinIO e documentação interativa de interfaces através do Flasgger (OpenAPI/Swagger).

A data e a hora de cada entrega foram obtidas diretamente a partir do histórico de controlo de versões com o comando `git log --all` formatado e posteriormente validadas na interface do GitHub. As evidências reunidas no documento comprovam os **commits** correspondentes, os ficheiros de configuração (`docker-compose.yml`), a documentação do projeto com a menção formal ao docente responsável (@siriani) e as interfaces dos serviços em funcionamento local.

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.	07/08/2026		não entregue
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.	20/08/2026	20/08/2026 23:55	entregue
3	Desacoplando o login — microserviço de autenticação	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.	28/08/2026	27/08/2026 20:37	entregue
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	02/09/2026 22:35	entregue
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	09/09/2026 20:36	entregue
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.	02/10/2026	02/10/2026 19:37	entregue
E1	Documentação Swagger/OpenAPI	Swagger UI com pelo menos 2 serviços documentados e “Try it out” executado.	sem prazo	02/10/2026 19:37	entregue

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.
Data planejada	07/08/2026
Data realizada	
Situação	não entregue
Evidência	
Link	—

O que foi feito. A atividade não foi realizada devido à minha ausência na aula, data em que a prática presencial de nivelamento foi conduzida em sala.

Dificuldades e como foram resolvidas. Não aplicável, em razão da não realização da atividade por falta justificada/ausência na data correspondente.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.
Data planejada	20/08/2026
Data realizada	20/08/2026 23:55
Situação	entregue
Evidência	Commit a97b538
Link	https://github.com/fezinhopatta/Tom-Hanks-api

O que foi feito. Foi desenvolvida uma aplicação web em Python/Flask integrada à API pública do TMDb para listar e buscar a filmografia do ator Tom Hanks. O sistema implementa uma camada de persistência conectada ao banco de dados externo MariaDB da disciplina, permitindo que cada usuário autenticado salve e gerencie sua própria lista de filmes favoritos com segregação de dados.

Para garantir o isolamento e a integridade das preferências, a modelagem relacional vinculou os identificadores dos filmes do TMDb diretamente às contas de usuário no banco de dados. Além disso, a aplicação foi estruturada para tratar as requisições HTTP da API externa e renderizar as informações dos títulos de forma dinâmica.

```

commit
1 parent: bec32b8 commit: ecab58c
11 files changed
+377 -2

.env.example
1 + TMDb_API_KEY=sua_chave_aqui
2 + DB_HOST=35.226.64.52
3 + DB_USER=usuario_disciplina
4 + DB_PASSWORD=senha_disciplina
5 + DB_NAME=banco_disciplina
6 + SECRET_KEY=chave_flask
7 + PORTA_ALUNO=5000

.gitignore
1 + .env
2 + __pycache__
3 + venv/
4 + *.pyc

```

```

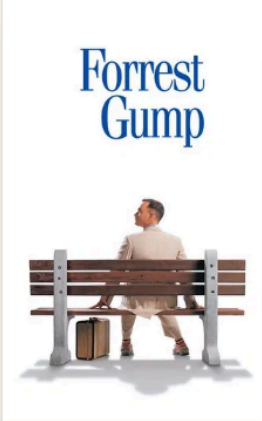
Tom-Hanks-api / README.md
fezinhopatta commit
ecab58c · 2 months ago History
Preview Code Blame 3 lines (3 loc) · 157 Bytes
Catálogo de Filmes - Tom Hanks
Aplicação que consome a API do TMDb e salva favoritos no MariaDB externo da disciplina. Professor responsável: @siriani

```

UMA CARREIRA EM CENAS

Os filmes de Tom Hanks

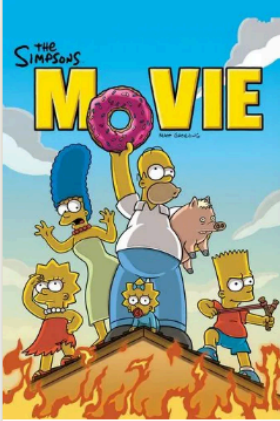
Explore os papéis mais marcantes, guarde seus favoritos e compartilhe suas impressões.



Forrest Gump

A man with a low IQ has accomplished great things in his life and been present during significant historic events—in each case, far...

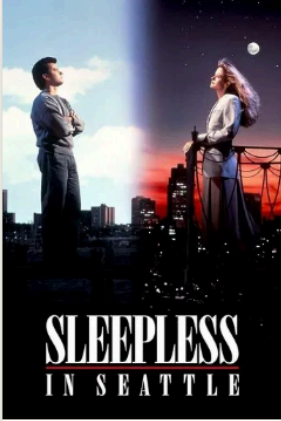
★ Favoritar



The Simpsons Movie

After Homer accidentally pollutes the town's water supply, Springfield is encased in a gigantic dome by the EPA and the Simpsons are...

★ Favoritar



Sleepless in Seattle

After the death of his mother, a young boy calls a radio station in an attempt to set his father up on a date. Across the country, an...

★ Favoritar



Saving Private Ryan

As U.S. troops storm the beaches of Normandy, three brothers lie dead on the battlefield, with a fourth trapped behind enemy lines. Range...

★ Favoritar

Dificuldades e como foram resolvidas. A principal dificuldade envolveu gerenciar a comunicação com o MariaDB externo e manter a segregação correta dos favoritos por usuário, resolvida com a correta parametrização das variáveis de conexão, criação de chaves estrangeiras apropriadas no esquema relacional e tratamento dos dados retornados pela API externa.

4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	27/08/2026 20:37
Situação	entregue
Evidência	GitHub — commit + docker-compose.yml + print do login funcionando
Link	https://github.com/fezinhopatta/Tom-Hanks-api/tree/Novo_Login

O que foi feito. Foi realizado o desacoplamento da lógica de autenticação do monólito original através da criação de um microsserviço dedicado (auth-service), orquestrado via Docker Compose em uma rede bridge interna (app-network). Para garantir a segurança da infraestrutura, apenas o container do catálogo expõe porta de acesso ao exterior, mantendo o microsserviço de autenticação totalmente isolado do host e comunicando-se exclusivamente por requisições HTTP internas entre containers.

Além da separação de responsabilidades e da gestão de papéis de usuário (admin e usuario), foi implementado o fluxo completo de recuperação de senha com tokens UUID únicos persistidos no MariaDB, dotados de uso único e expiração de 30 minutos. O envio real dos e-mails com as instruções e links de redefinição foi integrado com a API/SMTP do Brevo, permitindo o recebimento efetivo na caixa de entrada do usuário final.

Commits

The image shows a commit history interface. At the top, there's a filter bar with 'Novo_Login' selected and buttons for 'All users' and 'All time'. Below, a list of commits is shown, with one commit selected: 'Ajeitge: Email' by 'cooperativagorzos-gif' committed on Aug 27. The commit hash '3d9864c' is visible. Below the list, the details of the selected commit are shown, including the commit message 'Ajeitge: Email', the parent commit 'a53f9a6', and a list of changed files: 'Login_Novo_3', 'Novo_Login', and 'Sistema_Logs'. The interface also shows a search bar and a file explorer on the left.

Commits on Aug 27, 2026

Ajeitge: Email
cooperativagorzos-gif committed on Aug 27

3d9864c

Commit 3d9864c
cooperativagorzos-gif committed on Aug 27

Ajeitge: Email

1 parent a53f9a6 commit 3d9864c

2 files changed

app.py

The mockup shows a password recovery interface. At the top, it says 'TOM HANKS / RECUPERAÇÃO' in orange. Below that is a large heading 'Esqueceu sua senha?' in dark blue. A subtext in blue says 'Informe seu e-mail cadastrado para receber o link de redefinição.' There is an input field for the email with the placeholder 'seuemail@exemplo.com'. Below the input field is an orange button with the text 'Enviar Link de Recuperação'. At the bottom, there is a link 'Voltar ao login' in orange.

Dificuldades e como foram resolvidas. A principal dificuldade envolveu configurar a comunicação interna entre containers sem expor portas públicas e implementar a entrega real de e-mails via Brevo com persistência dos tokens; isso foi solucionado configurando as variáveis de ambiente e autenticação SMTP do Brevo no microserviço e modelando a tabela `reset_tokens` para validar prazo de expiração e bloqueio de reuso no MariaDB.

4.4 Atividade 4 — Controle de acesso por papel — RBAC

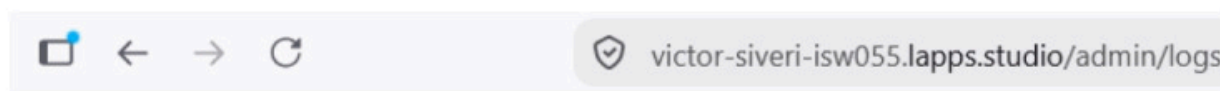
Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	02/09/2026 22:35
Situação	entregue
Evidência	GitHub — commit + print do 403 e da ação de admin
Link	https://github.com/fezinhopatta/Tom-Hanks-api/tree/Login_Novo_3

O que foi feito. Foi implementado o controle de acesso baseado em papéis (RBAC) com segregação entre usuários comuns e administradores. O backend do catálogo passou a validar ativamente as permissões para ações sensíveis, definindo a moderação e exclusão de comentários de terceiros como privilégio exclusivo do papel de administrador. Caso um usuário comum tente forçar a exclusão de um comentário alheio diretamente via rota ou requisição avulsa, o sistema bloqueia a operação retornando o código HTTP 403 Forbidden.

A arquitetura adotou o padrão de enforcement centralizado, no qual o serviço de catálogo realiza chamadas HTTP internas via Docker network para o microserviço de autenticação (/api/check-role) com o intuito de verificar e validar a role do usuário no banco de dados. Na interface, a renderização dos botões de ação também foi condicionada ao papel validado, exibindo as opções de exclusão universal apenas para moderadores.

Commits

 Login_Novo_3	 All users	 All time
Commits on Sep 3, 2026		
Revert "chore: altera fluxo de envio de e-mail para sincrónico e adiciona logs no frontend para debug"  cooperativagorzos-gif committed on Sep 3 f01b6cd  <>		
chore: altera fluxo de envio de e-mail para sincrónico e adiciona logs no frontend para debug  cooperativagorzos-gif committed on Sep 3 a673b3b  <>		
feat: adiciona suporte a variável APP_URL para envio de e-mails via Brevo  cooperativagorzos-gif committed on Sep 3 5a5d51e  <>		
Feat: Implementa RBAC (Atividade 4) - Padrões de acesso e Admin enforcement  cooperativagorzos-gif committed on Sep 2 cb2cd56  <>		
Commits on Aug 27, 2026		



Logs de Auditoria (Acesso Restrito: Admin)

Data/Hora (ms)	Usuário ID / Email
05/10/2026, 20:17:40	5
05/10/2026, 20:17:25	admin@teste.com
05/10/2026, 20:16:59	3
05/10/2026, 20:11:26	3
05/10/2026, 20:11:11	3
05/10/2026, 20:11:00	5

Dificuldades e como foram resolvidas. A principal dificuldade foi assegurar que o bloqueio não dependesse apenas do ocultamento visual de elementos no frontend e coordenar a checagem síncrona entre microsserviços; a resolução consistiu em implementar o enforcement diretamente nas rotas do catalog-service por meio de requisições internas ao auth-service, garantindo o retorno estrito de HTTP 403 para requisições não autorizadas.

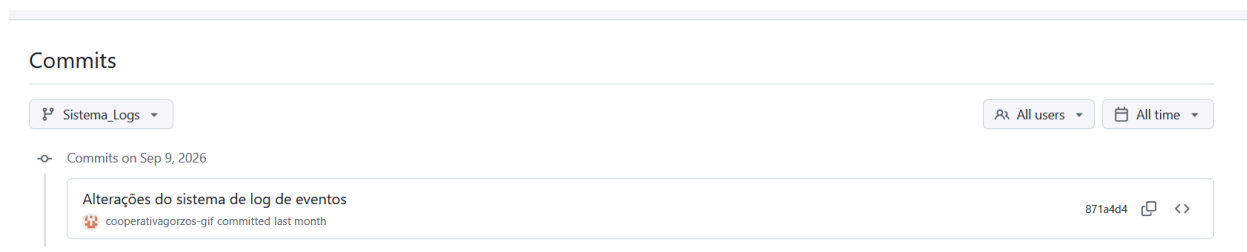
4.5 Atividade 5 — Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	09/09/2026 20:36

Situação	entregue
Evidência	GitHub — commit + print da consulta de logs pelo admin
Link	https://github.com/fezinhopatta/Tom-Hanks-api/tree/Sistema_Logs

O que foi feito. Foi desenvolvido um microserviço independente (log-service) integrado a uma instância de Redis na rede Docker interna para gerenciar eventos e logs de auditoria em memória com alta performance. A separação garantiu que operações frequentes de escrita não sobrecarregassem o MariaDB, armazenando ocorrências como login, logout, inclusão de favoritos, criação de comentários, ações de moderação e tentativas não autorizadas com código HTTP 403.

A persistência temporal dos eventos foi implementada utilizando Redis Streams por meio dos comandos XADD para inserção e XREVRANGE para leitura cronológica eficiente. Adicionalmente, foi criado um endpoint administrativo seguro no catálogo (/admin/logs) que consome o log-service para exibir os registros consolidados exclusivamente para usuários com papel de administrador.



Logs de Auditoria (Acesso Restrito: Admin)

Data/Hora (ms)	Usuário ID / Email	Ação	IP de Origem
05/10/2026, 20:25:53	5	login	172.28.0.6
05/10/2026, 20:25:35	3	logout	172.28.0.1
05/10/2026, 20:21:42	3	403_acessar_logs	172.28.0.1
05/10/2026, 20:21:13	3	login	172.28.0.6
05/10/2026, 20:21:06	5	logout	172.28.0.1
05/10/2026, 20:17:40	5	login	172.28.0.6
05/10/2026, 20:17:25	admin@teste.com	tentativa_login_falha	172.28.0.6
05/10/2026, 20:16:59	3	logout	172.28.0.1
05/10/2026, 20:11:26	3	403_acessar_logs	172.28.0.1
05/10/2026, 20:11:11	3	login	172.28.0.6
05/10/2026, 20:11:00	5	logout	172.28.0.1
05/10/2026, 20:10:30	5	login	172.28.0.6

Dificuldades e como foram resolvidas. A principal dificuldade envolveu estruturar o pipeline de auditoria assíncrono/desacoplado entre os serviços e garantir que a leitura no Redis retornasse os eventos de forma ordenada e legível; a solução adotada foi o uso do Redis Streams para registrar metadados com timestamp nativo e a criação de uma rota administrativa com controle RBAC para expor a listagem tratada apenas para perfis autorizados.

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.
Data planejada	02/10/2026

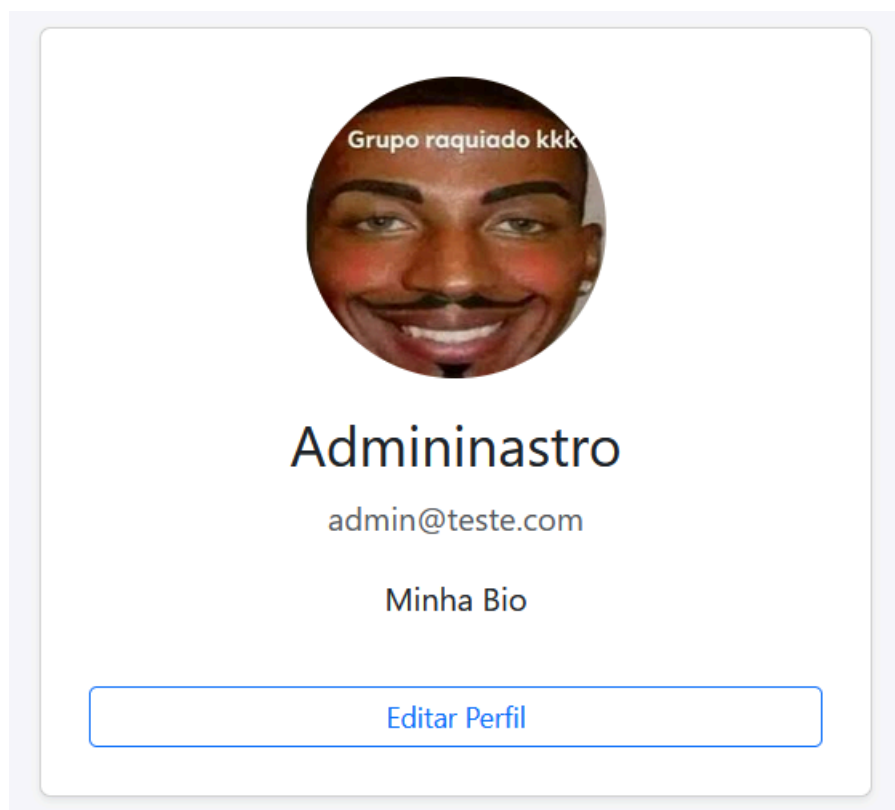
Data realizada	02/10/2026 19:37
Situação	entregue
Evidência	GitHub — commit + print do perfil com foto
Link	https://github.com/fezinhopatta/Tom-Hanks-api

O que foi feito. Foi implementada a página de perfil de usuário com suporte a biografia, listagem de filmes favoritos e upload de foto de avatar integrada a um Object Storage (MinIO) dedicado na rede Docker. Para preservar o desempenho e escalabilidade do banco relacional MariaDB, evitou-se o armazenamento de binários pesados (BLOBs), gravando apenas a URL de referência da imagem no esquema de dados.

Como decisão de arquitetura, optou-se pela configuração de um bucket com leitura pública em vez de URLs pré-assinadas (Presigned URLs), visando otimizar a performance de carregamento via cache de navegadores e dispensar overhead de re-assinatura e controle de expiração constante por se tratar de fotos públicas de perfil. Além disso, a rota de atualização (/perfil/) foi protegida por validação estrita de identidade, garantindo que requisições de edição feitas por usuários diferentes do proprietário da conta sejam rejeitadas com erro 403 Forbidden.

Commits

main	All users	All time
Commits on Oct 2, 2026		
hotfix: MiniO porta? fezinhopatta committed 3 days ago	500dd71	
hotfix: container name fezinhopatta committed 3 days ago	f6d4379	
Ajuste de perfils fezinhopatta committed 3 days ago	4f1f98d	



Dificuldades e como foram resolvidas. A principal dificuldade consistiu em fazer o endereço/IP público do MinIO funcionar corretamente para o utilizador final, uma vez que as imagens enviadas ficavam acessíveis internamente no contentor, mas falhavam ao serem carregadas pelo navegador do cliente externo. O problema foi solucionado fazendo com que o frontend/serviço intermediasse e fornecesse o IP público acessível da máquina anfitriã ao utilizador, garantindo a renderização correta da imagem sem conflitos de rede interna do Docker.

5 Atividades extras (opcional)

5.1 Atividade E1 — Documentação Swagger/OpenAPI

Descrição	Swagger UI com pelo menos 2 serviços documentados e “Try it out” executado.
Data planejada	sem prazo
Data realizada	02/10/2026 19:37
Situação	entregue
Evidência	Repositório público no GitHub — commit com data e hora, README mencionando github.com/siriani, print do sistema rodando
Link	https://github.com/fezinhopatta/Tom-Hanks-api

O que foi feito. Foi integrada a documentação interativa baseada no padrão OpenAPI/Swagger aos microserviços auth-service e log-service utilizando a biblioteca Flasgger. A interface Swagger UI foi configurada em ambos os serviços, permitindo inspecionar esquemas de requisição e resposta, além de realizar testes diretos nos endpoints através do recurso “Try it out”.

Para viabilizar o acesso direto aos painéis interativos a partir do navegador da máquina host, as portas 5001 e 5002 foram temporariamente expostas no docker-compose.yml. Além da navegação visual, as especificações brutas em formato JSON foram disponibilizadas via rotas dedicadas (/apispec_1.json) para integração com ferramentas externas de API.

Commits

 main ▾

 Commits on Oct 2, 2026

hotfix: MiniO porta?

 fezinhopatta committed 3 days ago

hotfix: container name

 fezinhopatta committed 3 days ago

Ajuste de perfils

 fezinhopatta committed 3 days ago

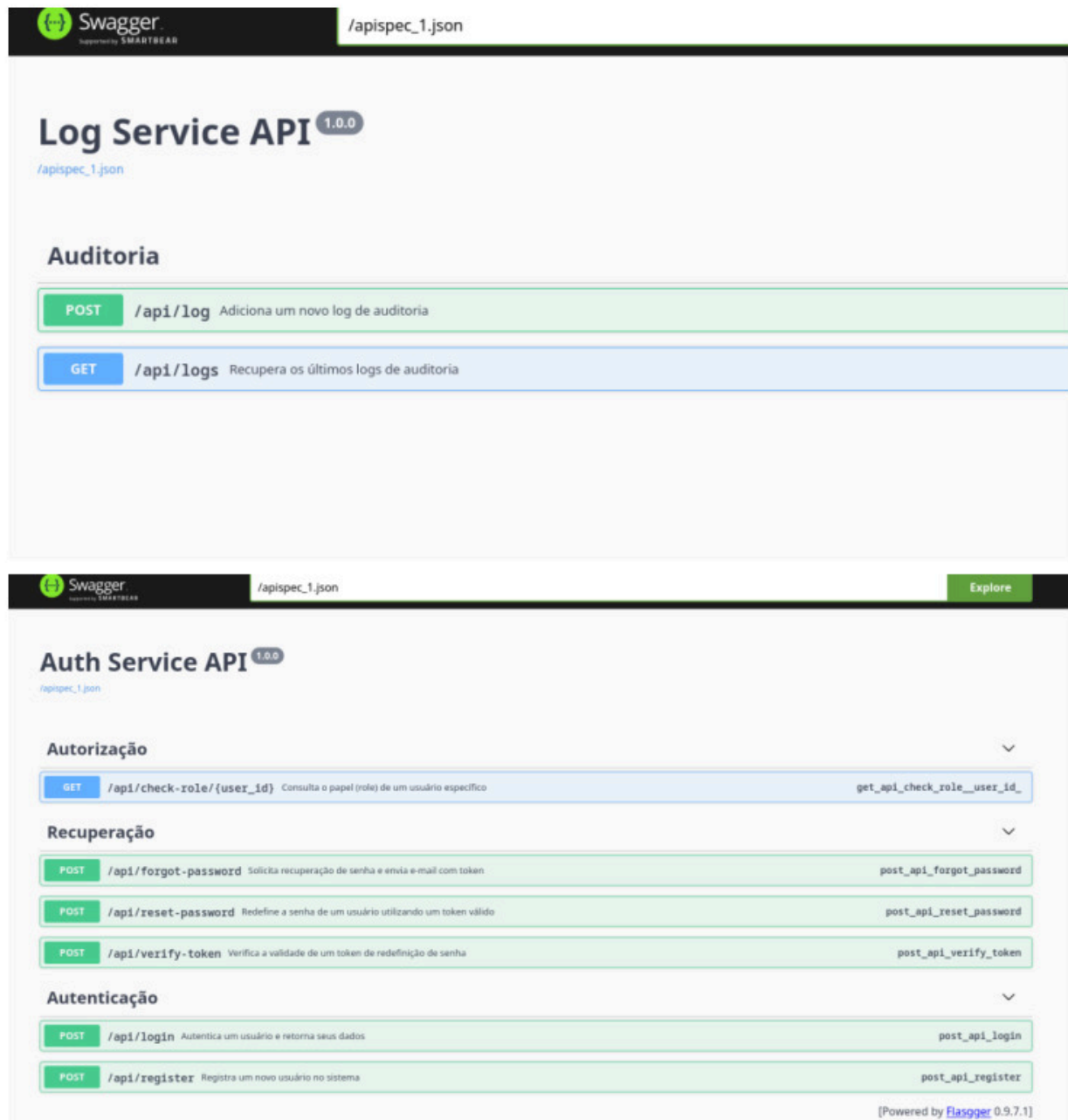
Auditoria com Redis)

Aplicação que consome a API do TMDB, exhibe a filmog
um microserviço de autenticação desacoplado.

Professor responsável: [@siriani](#)



O que mudou nesta Atividade (



Dificuldades e como foram resolvidas. A principal dificuldade envolveu documentar múltiplos serviços isolados na rede interna sem quebrar a segurança padrão e viabilizar os testes via host; isso foi solucionado mapeando temporariamente as portas dedicadas para cada microserviço no Docker Compose e configurando as anotações do Flasgger para gerar as especificações e a interface Swagger UI corretamente.

6 Considerações finais

Ao longo do desenvolvimento deste projeto prático, consolidei conhecimentos essenciais sobre a transição de um sistema monolítico para uma arquitetura orientada a microserviços com Docker Compose. O aprendizado abrangeu desde a segregação de responsabilidades na camada de autenticação até o uso de tecnologias especializadas para demandas pontuais, como a persistência de logs em memória via Redis

Streams, o armazenamento desacoplado de mídias em Object Storage com MinIO e a aplicação de controle de acesso rigoroso por papéis (RBAC).

O maior desafio técnico consistiu em orquestrar a comunicação entre múltiplos contêineres na rede interna mantendo as restrições de segurança do Docker, especialmente na resolução do roteamento de rede para que o cliente externo conseguisse acessar as imagens hospedadas no MinIO e na validação centralizada de papéis sem comprometer a latência. Olhando para a arquitetura construída, em um cenário futuro eu implementaria o controle de acesso baseado em tokens JWT com claims de autorização (Padrão B) em vez de validação síncrona via rede interna, reduzindo o tráfego HTTP entre os serviços de catálogo e autenticação.

Para o próximo ciclo, pretendo aprofundar na automação de pipelines de CI/CD para containerização e na adoção de um API Gateway para unificar o roteamento público e a camada de segurança dos microserviços. As competências adquiridas em orquestração, integração de serviços distribuídos e documentação interativa com Swagger/OpenAPI servirão de alicerce fundamental para a construção de soluções mais resilientes e escaláveis.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

Victor Hugo Cano Siveri

Pompeia, 07/10/2026